

Git for Developers

Module 6 — Common Workflows

Team Branching Strategies: Centralized, Gitflow, and Trunk-Based

Module 6 — Agenda

Standard Workflows

- Centralized Workflow — the simplest approach
- Feature Branch Workflow — one branch per feature
- Gitflow Workflow — structured releases with develop and main

Modern Workflows

- Trunk-Based Development — fast iteration, feature flags
- Choosing the right workflow for your team

Part 1 — Centralized Workflow

Centralized Workflow

Everyone works directly on `main` (or `master`):

```
Developer A: commit —▶ push to main
Developer B:           pull —▶ commit —▶ push to main
Developer C:           pull —▶ commit —▶ push
```

Characteristics

- No feature branches — all work goes directly to `main`
- Simple: identical to how SVN works
- Small teams, solo projects, or very tight coordination required

Centralized Workflow — Day to Day

```
# Start your day
git pull

# Make changes, commit locally
git add .
git commit -m "feat: improve search performance"

# Before pushing, pull again in case someone else pushed
git pull --rebase      # keeps history linear

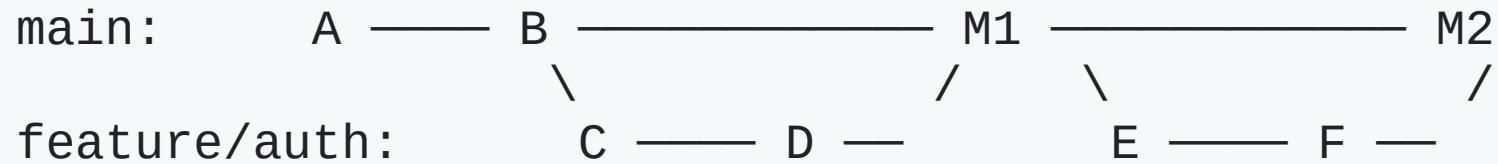
# Push to main
git push
```

Works fine for small scripts and solo projects. Does **not** scale to teams — no isolation, no review.

Part 2 — Feature Branch Workflow

Feature Branch Workflow

Each new feature or fix gets its own branch:



Rules

- `main` is always stable and deployable
- No direct commits to `main`
- Every change comes in through a **Pull Request**
- Branch is deleted after merging

Feature Branch Workflow — Day to Day

```
# 1. Start from an up-to-date main
git checkout main && git pull

# 2. Create a feature branch
git checkout -b feature/payment-integration

# 3. Work, commit locally
git add . && git commit -m "feat: add Stripe webhook handler"

# 4. Keep branch up to date with main
git fetch origin
git rebase origin/main

# 5. Push and open a Pull Request
git push -u origin feature/payment-integration
```

Pull Request Lifecycle

```
Developer: push feature branch
|
GitHub/GitLab: open Pull Request
|
Reviewers: review code, leave comments
|
Developer: address feedback, push more commits
|
CI/CD: run tests, linting, security scans
|
Reviewer: approve
|
Merge to main → delete branch → deploy
```

Part 3 — Gitflow Workflow

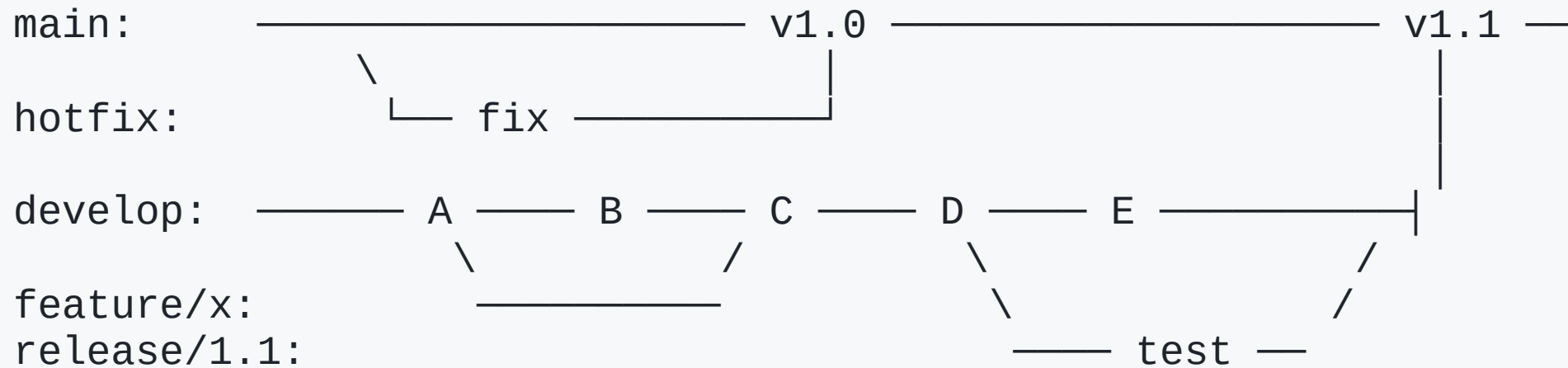
Gitflow — Overview

Gitflow defines a strict branching model with dedicated branches for every phase:

Branch	Purpose
<code>main</code>	Production-ready code — every commit is a release
<code>develop</code>	Integration branch — all features merge here
<code>feature/*</code>	New features — branch from <code>develop</code>
<code>release/*</code>	Release preparation — branch from <code>develop</code>
<code>hotfix/*</code>	Emergency fixes — branch from <code>main</code>

Gitflow was introduced by Vincent Driessen in 2010 and was widely adopted for scheduled release software.

Gitflow — Branch Flow



Gitflow — Feature Development

```
# Start a feature
git checkout develop
git checkout -b feature/user-notifications

# Work and commit on feature branch
git commit -m "feat: add email notification service"

# Merge feature back to develop
git checkout develop
git merge --no-ff feature/user-notifications
git branch -d feature/user-notifications
git push origin develop
```

Gitflow — Release Process

```
# Branch off develop when ready to release
git checkout develop
git checkout -b release/1.2.0

# Final tweaks, version bump, docs
git commit -m "chore: bump version to 1.2.0"

# Merge to main AND develop (keep them in sync)
git checkout main
git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m "Release 1.2.0"

git checkout develop
git merge --no-ff release/1.2.0
git branch -d release/1.2.0
```

Gitflow — Hotfix Process

```
# Production bug found – branch from main (not develop)
git checkout main
git checkout -b hotfix/payment-crash

# Fix the bug
git commit -m "fix: handle null payment token"

# Merge to both main and develop
git checkout main
git merge --no-ff hotfix/payment-crash
git tag -a v1.1.1 -m "Hotfix 1.1.1"

git checkout develop
git merge --no-ff hotfix/payment-crash
git branch -d hotfix/payment-crash
```


Gitflow Tooling

```
# Install git-flow CLI extension
brew install git-flow-avh

# Initialize Gitflow in your repo
git flow init

# Start, finish features/releases/hotfixes with shortcuts
git flow feature start user-notifications
git flow feature finish user-notifications

git flow release start 1.2.0
git flow release finish 1.2.0

git flow hotfix start payment-crash
git flow hotfix finish payment-crash
```

Part 4 — Trunk-Based Development

Trunk-Based Development (TBD)

Everyone commits **directly to** `main` (the "trunk") or uses very **short-lived branches** (1-2 days max):

```
main (trunk):  A — B — C — D — E — F — G    ← continuous integration
                |           |
                feature flag  feature flag
                (off in prod) (on in prod)
```

Core principles

- Small, frequent commits to `main`
- Feature flags hide incomplete work in production
- No long-lived branches
- Automated tests must pass before every merge

Feature Flags in TBD

Instead of long-lived branches, incomplete features are hidden with flags:

```
// Feature flag pattern
const NEW_AUTH_FLOW = process.env.FEATURE_NEW_AUTH === 'true';

function handleLogin(req, res) {
  if (NEW_AUTH_FLOW) {
    return newAuthService.login(req, res); // new implementation
  }
  return legacyAuthService.login(req, res); // stable fallback
}
```

Enable the flag in staging/dev; disable in prod until the feature is complete and tested.

Trunk-Based Development — Day to Day

```
# Work directly on main (for very small changes)
git pull
git commit -m "feat: improve query performance"
git push

# For larger changes: short-lived feature branch (max 1-2 days)
git checkout -b feat/search-index
# ... work ...
git push -u origin feat/search-index
# Open PR, get review, merge within 24-48 hours
# Delete branch
```

Part 5 — Choosing the Right Workflow

Workflow Comparison

Factor	Centralized	Feature Branch	Gitflow	Trunk-Based
Team size	Solo/tiny	Small–medium	Medium–large	Any
Release cadence	Continuous	Flexible	Scheduled releases	Continuous
Branch complexity	None	Low	High	Very low
Code review	Optional	Pull Requests	Pull Requests	Required (automated)
CI/CD maturity	Low	Medium	Medium	High

When to Use Each Workflow

Centralized

- Solo developer, scripts, learning projects

Feature Branch

- Most SaaS teams — a great default choice
- Good balance of structure and simplicity

Gitflow

- Products with **scheduled, versioned releases** (desktop software, SDKs, APIs with SLAs)
- Teams that need to maintain multiple active release lines simultaneously

Trunk-Based Development

- High-performing teams with strong CI/CD and testing discipline
- Products with **continuous deployment** to production

Summary — Workflows

Workflow	Key idea	Branch model
Centralized	One branch	<code>main</code> only
Feature Branch	One branch per feature	<code>main</code> + <code>feature/*</code>
Gitflow	Release-oriented	<code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>
Trunk-Based	Continuous integration	<code>main</code> + short-lived branches + feature flags

No workflow is universally best. Choose based on team size, release cadence, and CI/CD maturity.